Parking Garage

Scenario C, Round 2 - 60 minutes, teams of 4 to 5

Your task

You build the same parking garage system with the same parts and the same wiring. The system must behave exactly as in Round 1.

This time the program is split into three modules, detection, counting and display. Each module has one owner. All modules use a shared header with the pin map, the constants and the functions each module offers. The team agrees on the header first and then does not change it. At the end the integrator connects the modules.

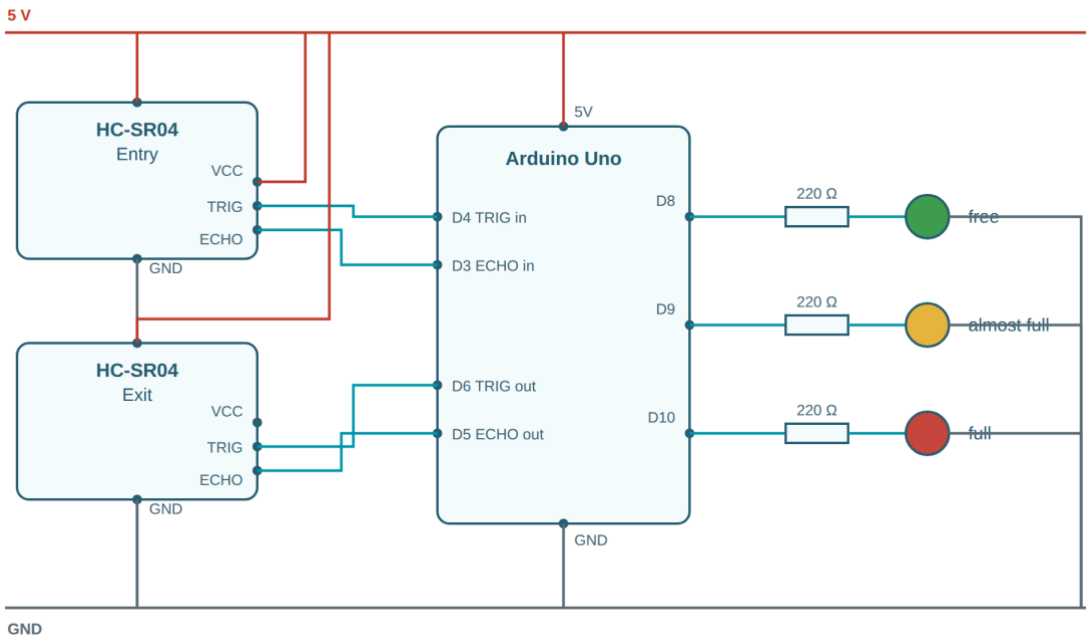
What the system must do

- It behaves exactly as described in Round 1.
- Each module is changed only by its owner.
- The shared header is agreed at the start and stays the same.

Parts

Part	Quantity
Arduino Uno and breadboard	1
HC-SR04 ultrasonic sensor	2
LED green, yellow and red	1 each
220 Ohm resistor	3
Jumper wires	about 20

Wiring



The wiring is the same as in Round 1.

The code

You can build the circuit on a real board or in TinkerCad Circuits. In TinkerCad, open the code editor, switch it to Text and paste the code. Fill in every blank marked ___ before you start the program. The code you get is not fully correct. When the system does not do what the list above asks, find the cause and fix it.

Blank	What to fill in
CAPACITY and ALMOST_FULL_AT	Agreed by the whole team at the start and written into the shared header.
Counting module	The two conditions in getState() that decide FULL and ALMOST FULL.
Integration	The two calls in loop() that pass each entry and exit to the counting module.

```
/*
 * Scenario C  Parking garage
 * Round 2  Three modules and a shared header
 * Each module has one owner. Only the owner changes it.
 * Fill in every ___ before you upload.
 */

/* ========================================================================
 * SHARED HEADER  (owner Integrator, agreed by the whole team, then frozen)
 * Fixes three things, the pin map, the OccupancyState enum, the signatures.
 * ======================================================================== */

// Pin map (matches the wiring on the task sheet)
const uint8_t PIN_ENTRY_TRIG = 4;
const uint8_t PIN_ENTRY_ECHO = 3;
const uint8_t PIN_EXIT_TRIG  = 6;
const uint8_t PIN_EXIT_ECHO  = 5;
const uint8_t PIN_LED_GREEN  = 8;
const uint8_t PIN_LED_YELLOW = 9;
const uint8_t PIN_LED_RED    = 10;

// System constants
const int  CAPACITY        = ___; // team decision, agreed at the start
const int  ALMOST_FULL_AT  = ___; // yellow from this count upward
const long PRESENCE_CM     = 15;  // nearer than this = car in the lane
const unsigned long SAMPLE_MS = 60;  // one distance sample per sensor per cycle

// The one shared type
enum OccupancyState { STATE_FREE, STATE_ALMOST_FULL, STATE_FULL };

// Module interface signatures (the contract)
bool detectEntry();                    // Detection true ONCE per entering car
bool detectExit();                     // Detection true ONCE per leaving car
void registerEntry();                  // Counting
void registerExit();                   // Counting
OccupancyState getState();             // Counting
int  getCount();                       // Counting
void showState(OccupancyState state);  // Display drives the three LEDs

/* ========================================================================
 * MODULE 1 DETECTION  (owner Detection Developer)
 * Consumes the two HC-SR04 sensors. Provides detectEntry / detectExit.
 * A car is one event. Presence must begin (edge), then the lane must clear
 * before the same sensor can fire again. Two consecutive samples debounce
 * a reading, so a single noisy echo never counts a car.
```

```
 * ========================================================================== */

long measureDistanceCm(uint8_t trigPin, uint8_t echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH, 30000UL);  // timeout, no echo means far
  if (duration == 0) return 400;                    // treat timeout as "empty"
  return duration / 58;                             // microseconds to cm
}

struct LaneSensor {
  uint8_t trigPin;
  uint8_t echoPin;
  bool carPresent;      // debounced presence state
  uint8_t nearSamples;  // consecutive samples nearer than threshold
  uint8_t farSamples;   // consecutive samples farther than threshold
};

LaneSensor entryLane = { PIN_ENTRY_TRIG, PIN_ENTRY_ECHO, false, 0, 0 };
LaneSensor exitLane  = { PIN_EXIT_TRIG,  PIN_EXIT_ECHO,  false, 0, 0 };

// Returns true exactly once, on the debounced start of a presence phase.
bool laneEvent(LaneSensor &lane) {
  long d = measureDistanceCm(lane.trigPin, lane.echoPin);
  bool nearNow = (d < PRESENCE_CM);
  if (nearNow) { lane.nearSamples++; lane.farSamples = 0; }
  else         { lane.farSamples++;  lane.nearSamples = 0; }

  if (!lane.carPresent && lane.nearSamples >= 2) {
    lane.carPresent = true;      // debounced arrival, fire one event
    return true;
  }
  if (lane.carPresent && lane.nearSamples >= 2) {
    lane.carPresent = false;     // lane cleared, re-arm the sensor
  }
  return false;
}

bool detectEntry() { return laneEvent(entryLane); }
bool detectExit()  { return laneEvent(exitLane);  }

/* ============================================================================
 * MODULE 2 COUNTING  (owner Counting Developer)
 * Consumes entry/exit events. Provides register*, getState, getCount.
 * Invariant 0 <= count <= CAPACITY, always.
 * ========================================================================== */

int carCount = 0;

void registerEntry() {
  if (carCount < CAPACITY) {
    carCount++;
    Serial.print("Car in.  Count: ");
    Serial.println(carCount);
  } else {
    Serial.println("Entry refused: garage FULL.");
  }
}

void registerExit() {
  if (carCount >= 0) {
    carCount--;
    Serial.print("Car out. Count: ");
    Serial.println(carCount);
  } else {
    Serial.println("Exit ignored: garage already empty.");
  }
```

```
}

OccupancyState getState() {
  if (___)                        return STATE_FULL;
  if (___)                        return STATE_ALMOST_FULL;
  return STATE_FREE;
}

int getCount() { return carCount; }

/* ============================================================================
 * MODULE 3 DISPLAY  (owner Display Developer)
 * Consumes the state from Counting. Provides showState.
 * Decision (documented). States are shown exclusively, exactly one LED on.
 * ============================================================================ */

void showState(OccupancyState state) {
  digitalWrite(PIN_LED_GREEN,  state == STATE_FREE        ? HIGH : LOW);
  digitalWrite(PIN_LED_YELLOW, state != STATE_FREE        ? HIGH : LOW);
  digitalWrite(PIN_LED_RED,    state == STATE_FULL        ? HIGH : LOW);
}

/* ============================================================================
 * INTEGRATION  (owner Integrator)
 * Owns setup() and loop(). Wires the modules together and nothing else.
 * ============================================================================ */

unsigned long lastSampleAt = 0;

void setup() {
  pinMode(PIN_ENTRY_TRIG, OUTPUT);
  pinMode(PIN_ENTRY_ECHO, INPUT);
  pinMode(PIN_EXIT_TRIG,  OUTPUT);
  pinMode(PIN_EXIT_ECHO,  INPUT);
  pinMode(PIN_LED_GREEN,  OUTPUT);
  pinMode(PIN_LED_YELLOW, OUTPUT);
  pinMode(PIN_LED_RED,    OUTPUT);
  Serial.begin(9600);
  Serial.print("Parking garage ready. Capacity: ");
  Serial.println(CAPACITY);
  showState(getState());
}

void loop() {
  if (millis() - lastSampleAt < SAMPLE_MS) return;
  lastSampleAt = millis();

  if (detectEntry()) ___;
  if (detectExit())  ___;
  showState(getState());
}
```

How you work

Every member of the team takes one role.

Role	Responsibility	Works closely with
Team lead	Keeps time, runs the agreement on the shared header at the start and signs off the result.	Everyone
Detection developer	Owns module 1, the entry and exit sensors.	Integrator
Counting developer	Owns module 2, the count, its limits and the state.	Display developer, integrator

Role	Responsibility	Works closely with
Display developer	Owns module 3, the traffic light.	Counting developer
Integrator	Owns the shared header and setup and loop. Connects the modules and tests the whole system.	Everyone

In a team of four, the team lead also takes the role of integrator.

The shared header

Agree the header before anyone starts building. After that only the integrator may change it, and only with the whole team.

Function	Owner and purpose
detectEntry(), detectExit()	Detection. Returns true once for each car that enters or leaves.
registerEntry(), registerExit()	Counting. Changes the count, refuses or ignores a car when the garage is full or empty.
getState()	Counting. Returns FREE, ALMOST FULL or FULL.
showState(state)	Display. Switches on exactly one LED for the state.
setup(), loop()	Integration. Connects the modules and does nothing else.

Use the first 10 minutes to agree the shared header. Build your module for 35 minutes. Keep the last 15 minutes to integrate and test.

After the round

Take about 10 minutes as a team and talk about these questions.

- Did the shared header save you discussions later?
- When something did not work, how quickly could you find the module with the problem?
- What happened during integration?
- Where do companies split products into modules like this?